Document Number:	**P0194R6**, ISO/IEC JTC1 SC22 WG21
Audience:	CWG, LWG
Date:	2018-03-16
Authors:	Matus Chochlik (chochlik@gmail.com)
	Axel Naumann (axel@cern.ch)
	David Sankel (dsankel@bloomberg.net)

Static reflection

0. Introduction

This paper proposes to add support for compile-time reflection as a Technical Specification for C++. We propose that the compiler shall generate meta-object types — representations of certain program declarations, which can be reasoned about at compile time. These meta-object types can then be used, through a set of operations to obtain various pieces of metadata, like declaration names, lists of scope members, information about specifiers, and so on. The meta-object types are implemented as unique, unnamed types conforming to defined concepts, and the operations are implemented as class templates.

This paper is accompanied by two more papers — P0385, which discusses the use cases, rationale, design decisions, future evolution of the proposed reflection facility and contains multiple examples of use and replies to frequently asked questions; and P0578 which gives a concise introduction to the feature set presented here.

Reflection proposed here behaves as follows:

```
enum E { first, second };
using E_m = reflexpr(E);
using namespace std::experimental::reflect;
using first_m = get_element_t<0, get_enumerators_t<E_m>>;
std::cout << get_name_v<first_m> << std::endl; // prints "first"
```

The publication as a Technical Specification is intended to collect implementation and usage experience on several aspects of this proposal. One of these aspects is the unspecified nature of the meta-object type's uniqueness for subsequent invocations of the `reflexpr` operator on the same operand.

0.1 Interplay with other proposals

This proposal relies on the Concepts TS rebased onto C++17. It also assumes that WG21 will incorporate proper compile-time strings; until then, this proposal provides a placeholder implementation (see [reflect.ops.named]).

1 Scope [intro.scope]

This Technical Specification describes extensions to the C++ Programming Language (Clause 2) that enable operations on source code. These extensions include new syntactic forms and modifications to existing language semantics, as well as changes and additions to the existing library facilities.

The International Standard, ISO/IEC 14882, provides important context and specification for this Technical Specification. This document is written as a set of changes against that specification, as modified by ISO/IEC TS 19217:2015. Instructions to modify or add paragraphs are written as explicit instructions. Modifications made directly to existing text from the International Standard use underlining to represent added text and ~~strikethrough~~ to represent deleted text.

2 Normative references [intro.refs]

The following referenced document is indispensable for the application of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

- ISO/IEC 14882:2017, *Programming Languages - C++*
- ISO/IEC TS 19217:2015, *Programming Languages - C++ Extensions for Concepts*

ISO/IEC 14882:2017 is hereafter called the *C++ Standard*.

ISO/IEC TS 19217:2015 is hereafter called the *Concepts TS*.

The numbering of clauses, subclauses, and paragraphs in this document reflects the numbering in the C++ Standard and the Concepts TS. References to clauses and subclauses not appearing in this Technical Specification refer to the original, unmodified text in the Concepts TS, or in the C++ Standard for clauses and subclauses not appearing in the Concepts TS.

3 Terms and definitions [intro.defs]

No terms and definitions are listed in this document. ISO and IEC maintain terminological databases for use in standardization at the following addresses:

- IEC Electropedia: available at http://www.electropedia.org/
- ISO Online browsing platform: available at http://www.iso.org/obp

4 General [intro]

4.1 Implementation compliance [intro.compliance]

Conformance requirements for this specification are those defined in subclause 4.1 in the Concepts TS, except that references to the Concepts TS or the C++ Standard therein shall be taken as referring to the document that is the result of applying the editing instructions. Similarly, all references to the Concepts TS or the C++ Standard in the resulting document shall be taken as referring to the resulting document itself. [*Note:* Conformance is defined in terms of the behavior of programs. — *end note*]

4.2 Namespaces and headers [intro.namespaces]

Whenever a name `x` declared in subclause 21.11 at namespace scope is mentioned, the name `x` is assumed to be fully qualified as `::std::experimental::reflect::v1::x`, unless otherwise specified. The header described in this specification (see Table 1) shall import the contents of `::std::experimental::reflect::v1` into `::std::experimental::reflect` as if by:

```
namespace std::experimental::reflect {
  inline namespace v1 {}
}
```

Whenever a name `x` declared in the standard library at namespace scope is mentioned, the name `x` is assumed to be fully qualified as `::std::x`, unless otherwise specified.

Table 1 — Reflection library headers

<experimental/reflect>

4.3 Acknowledgements [intro.ack]

This work is the result of a collaboration of researchers in industry and academia. We wish to thank people who made valuable contributions within and outside these groups, including Ricardo Fabiano de Andrade, Roland Bock, Chandler Carruth, Klaim-Jol Lamotte, Jens Maurer, and many others not named here who contributed to the discussion.

5 Lexical conventions [lex]

5.1 Keywords [lex.key]

In C++ [lex.key], add the keyword `reflexpr` to the list of keywords in Table 5.

6 Basic concepts [basic]

In C++ [basic], add the following last paragraph:

> An *alias* is a name introduced by a `typedef` declaration, an *alias-declaration*, or a *using-declaration*.

6.1 Fundamental types [basic.fundamental]

In C++ [basic.fundamental], apply the following change:

> An expression of type *cv* `void` shall be used only as an expression statement (9.2), as an operand of a comma

> expression (8.19), as a second or third operand of `?:` (8.16), as the operand of `typeid`, `noexcept`, `reflexpr`, or `decltype`, as the expression in a return statement (9.6.3) for a function with the return type *cv* `void`, or as the operand of an explicit conversion to type *cv* `void`.

7 Standard conversions [conv]

No changes are made to Clause 7 of the C++ Standard.

8 Expressions [expr]

No changes are made to Clause 8 of the C++ Standard.

9 Statements [stmt.stmt]

No changes are made to Clause 9 of the C++ Standard.

10 Declarations [dcl.dcl]

10.1 Specifiers [dcl.spec]

10.1.7 Type specifiers [dcl.type]

10.1.7.2 Simple type specifiers [dcl.type.simple]

In C++ [dcl.type.simple], apply the following change

> The simple type specifiers are
>
> *simple-type-specifier*:
> *nested-name-specifier*$_{\text{opt}}$ *type-name*
> *nested-name-specifier* `template` *simple-template-id*
> `char`
> `char16_t`
> `char32_t`
> `wchar_t`
> `bool`
> `short`
> `int`
> `long`
> `signed`
> `unsigned`
> `float`
> `double`
> `void`
> `auto`
> *decltype-specifier*
> *constrained-type-specifier*
> *reflexpr-specifier*
> *type-name*:
> *class-name*
> *enum-name*
> *typedef-name*
> *simple-template-id*
> *decltype-specifier*:
> `decltype (` *expression* `)`
> `decltype ( auto )`
> *reflexpr-specifier*:
> `reflexpr (` *reflexpr-operand* `)`
>
> *reflexpr-operand*:
> `::`
> *type-id*
> *nested-name-specifier*$_{\text{opt}}$ *identifier*

nested-name-specifier$_{opt}$ *simple-template-id*

...

The other *simple-type-specifiers* specify either a previously-declared type, a type determined from an expression, a reflection meta-object type (10.1.7.6), or one of the fundamental types (6.9.1).

Add the following row to Table 11

reflexpr (*reflexpr-operand*)	The type as defined below

At the end of 10.1.7.2, insert the following paragraph:

For a *reflexpr-operand* **x**, the type denoted by `reflexpr(x)` is an implementation-defined type that satisfies constraints as laid out in 10.1.7.6.

10.1.7.6 Reflection type specifier **[dcl.type.reflexpr]**

Insert the following section:

The `reflexpr` operator yields a type `T` that allows inspection of some properties of its operand through type traits or type transformations on `T` (21.11.4). The operand to the `reflexpr` operator shall be a type, namespace, enumerator, variable, structured binding or data member. Any such `T` satisfies the requirements of `reflect::Object` (21.11.3) and other `reflect` concepts, depending on the operand. A type satisfying the requirements of `reflect::Object` is called a *meta-object type*. A meta-object type is an incomplete namespace-scope class type ([class]).

An entity or alias `A` is *reflection-related* to an entity or alias `B` if

- `A` and `B` are the same entity or alias,
- `A` is a variable or enumerator and `B` is the type of `A`,
- `A` is an enumeration and `B` is the underlying type of `A`,
- `A` is a class and `B` is a member or base class of `A`,
- `A` is a non-template alias that designates the entity `B`,
- `A` is a class nested in `B` (12.2.5 [class.nest]),
- `A` is not the global namespace and `B` is an enclosing namespace of `A`, or
- `A` is reflection-related to an entity or alias `X` and `X` is reflection-related to `B`.

[*Note:* This relationship is reflexive and transitive, but not symmetric. — *end note*]

[*Example:*

```
struct X;
struct B  {
   using X = ::X;
   typedef X Y;
};
struct D : B {
   using B::Y;
};
```

The alias `D::Y` is reflection-related to `::X`, but not to `B::Y` or `B::X`. — *end example*]

Zero or more successive applications of type transformations that yield meta-object types (21.11.4) to the type denoted by a *reflexpr-specifier* enable inspection of entities and aliases that are reflection-related to the operand; such a meta-object type is said to *reflect* the respective reflection-related entity or alias.

[*Example:*

```
template <typename T> std::string get_type_name() {
   namespace reflect = std::experimental::reflect;
   // T_t is an Alias reflecting T:
   using T_t = reflexpr(T);
   // aliased_T_t is a Type reflecting the type for which T is a synonym:
   using aliased_T_t = reflect::get_aliased_t<T_t>;
   return reflect::get_name_v<aliased_T_t>;
}

std::cout << get_type_name<std::string>(); // prints "basic_string"
```

— *end example*]

The type specified by the *reflexpr-specifier* is implementation-defined. It is unspecified whether repeatedly applying `reflexpr` to the same operand yields the same type or a different type. [*Note:* If a meta-object type reflects an

incomplete class type, certain type transformations (21.11.4) cannot be applied. — *end note*]

[*Example:*

```
class X;
using X1_m = reflexpr(X);
class X {};
using X2_m = reflexpr(X);
using X_bases_1 = std::experimental::reflect::get_base_classes_t<X1_m>; // ok, X1_m reflects complete class X
using X_bases_2 = std::experimental::reflect::get_base_classes_t<X2_m>; // ok
std::experimental::reflect::get_reflected_type_t<X1_m> x; // ok, type X is complete
```

— *end example*]

For the operand `::`, the type specified by the `reflexpr-specifier` satisfies `reflect::GlobalScope`. For an operand of the form *identifier* where *identifier* is a template *type-parameter*, the type satisfies both `reflect::Type` and `reflect::Alias`.

The *identifier* or *simple-template-id* is looked up using the rules for name lookup (6.4): if a *nested-name-specifier* is included in the operand, qualified lookup (6.4.3) of *nested-name-specifier identifier* or *nested-name-specifier simple-template-id* will be performed, otherwise unqualified lookup (6.4.1) of *identifier* or *simple-template-id* will be performed. The type specified by the *reflexpr-specifier* satisfies concepts depending on the result of the name lookup, as shown in Table 12.

Table 12 — `reflect` concept (21.11.3) that the type specified by a *reflexpr-specifier* satisfies, for a given *reflexpr-operand identifier* or *simple-template-id.*

Category	***identifier* or *simple-template-id* kind**	**`reflect` Concept**
type	*class-name* designating a union	`reflect::Record`
	class-name designating a non-union class	`reflect::Class`
	enum-name	`reflect::Enum`
	type-name introduced by a *using-declaration*	both `reflect::Type` and `reflect::Alias`
	any other *typedef-name*	both `reflect::Type` and `reflect::Alias`
namespace	*namespace-alias*	both `reflect::Namespace` and `reflect::Alias`
	any other *namespace-name*	both `reflect::Namespace` and `reflect::ScopeMember`
data member	the name of a data member	`reflect::Variable`
value	the name of a variable or structured binding that is not a local entity	`reflect::Variable`
	the name of an enumerator	both `reflect::Enumerator` and `reflect::Constant`

If the *reflexpr-operand* designates a *type-id* not explicitly mentioned in Table 12, the type represented by the *reflexpr-specifier* satisfies `reflect::Type`. Any other *reflexpr-operand* renders the program ill-formed.

If the *reflexpr-operand* designates an entity or alias at block scope (6.3.3) or function prototype scope (6.3.4), the program is ill-formed. If the *reflexpr-operand* designates a class member, the type represented by the *reflexpr-specifier* also satisfies `reflect::RecordMember`. If the *reflexpr-operand* designates an variable or a data member, it is an unevaluated operand (expr.context). If the *reflexpr-operand* designates both an alias and a class name, the type represented by the *reflexpr-specifier* reflects the alias and satisfies `Alias`.

11 Declarators [dcl.decl]

11.1 Type names [dcl.name]

To specify type conversions explicitly, and as an argument of `sizeof`, `alignof`, `new`, ~~or~~ `typeid`, or reflexpr, the name of a type shall be specified.

12 Classes [class]

No changes are made to Clause 12 of the C++ Standard.

13 Derived classes [class.derived]

No changes are made to Clause 13 of the C++ Standard.

14 Member access control [class.access]

No changes are made to Clause 14 of the C++ Standard.

15 Special member functions [special]

No changes are made to Clause 15 of the C++ Standard.

16 Overloading [over]

No changes are made to Clause 16 of the C++ Standard.

17 Templates [temp]

17.6.2.1 Dependent types [temp.dep.type]

A type is dependent if it is
[...]
— a *simple-template-id* in which either the template name is a template parameter or any of the template arguments is a dependent type or an expression that is type-dependent or value-dependent, ~~or~~
— denoted by `decltype(`*expression*`)`, where *expression* is type-dependent (14.6.2.2)~~.~~, or
— denoted by `reflexpr(`*operand*`)`, where *operand* designates a dependent type or a member of an unknown specialization.

18 Exception handling [except]

No changes are made to Clause 18 of the C++ Standard.

19 Preprocessing directives [cpp]

No changes are made to Clause 19 of the C++ Standard.

20 Library introduction [library]

20.5 Library-wide requirements [requirements]

20.5.1 Library contents and organization [organization]

20.5.1.2 Headers [headers]

Add <experimental/reflect> to Table 16 — C++ library headers.

21 Language support library [language.support]

Insert the following section:

21.11 Static reflection [reflect]

21.11.1 In general [reflect.general]

As laid out in 10.1.7.6, compile-time constant metadata, describing various aspects of a program (static reflection data), can be accessed through meta-object types. The actual metadata is obtained by instantiating templates constituting the interface of the meta-object types. These templates are collectively referred to as *meta-object operations*.

Meta-object types satisfy different concepts (21.11.3) depending on the type they reflect (10.1.7.6). These concepts can also be used for meta-object type classification. They form a generalization-specialization hierarchy, with `reflect::Object` being the common generalization for all meta-object types. Unary operations and type transformations used to query static reflection data associated with these concepts are described in 21.11.4.

21.11.2 Header `<experimental/reflect>` synopsis [reflect.synopsis]

```
namespace std::experimental::reflect {
```

```
  inline namespace v1 {

  // 21.11.3 Concepts for meta-object types
  template <class T> concept Object;
  template <class T> concept ObjectSequence;
  template <class T> concept Named;
  template <class T> concept Alias;
  template <class T> concept RecordMember;
  template <class T> concept Enumerator;
  template <class T> concept Variable;
  template <class T> concept ScopeMember;
  template <class T> concept Typed;
  template <class T> concept Namespace;
  template <class T> concept GlobalScope;
  template <class T> concept Class;
  template <class T> concept Enum;
  template <class T> concept Record;
  template <class T> concept Scope;
  template <class T> concept Type;
  template <class T> concept Constant;
  template <class T> concept Base;


  // 21.11.4 Meta-object operations
  // 21.11.4.1 Multi-concept operations
  template <class T> struct is_public;
  template <class T> struct is_protected;
  template <class T> struct is_private;

  template <class T>
    constexpr auto is_public_v = is_public<T>::value;
  template <class T>
    constexpr auto is_protected_v = is_protected<T>::value;
  template <class T>
    constexpr auto is_private_v = is_private<T>::value;

  // 21.11.4.2 Object operations
  template <Object T1, Object T2> struct reflects_same;
  template <class T> struct get_source_line;
  template <class T> struct get_source_column;
  template <class T> struct get_source_file_name;

  template <Object T1, Object T2>
    constexpr auto reflects_same_v = reflects_same<T1, T2>::value;
  template <class T>
    constexpr auto get_source_line_v = get_source_line<T>::value;
  template <class T>
    constexpr auto get_source_column_v = get_source_column<T>::value;
  template <class T>
    constexpr auto get_source_file_name_v = get_source_file_name<T>::value;

  // 21.11.4.3 ObjectSequence operations
  template <ObjectSequence S> struct get_size;
  template <size_t I, ObjectSequence S> struct get_element;
  template <template <class...> class Tpl, ObjectSequence S>
    struct unpack_sequence;

  template <ObjectSequence T>
    constexpr auto get_size_v = get_size<T>::value;
  template <size_t I, ObjectSequence S>
    using get_element_t = typename get_element<I, S>::type;
  template <template <class...> class Tpl, ObjectSequence S>
    constexpr auto unpack_sequence_t = unpack_sequence<Tpl, S>::type;

  // 21.11.4.4 Named operations
  template <Named T> struct is_unnamed;
  template <Named T> struct get_name;
  template <Named T> struct get_display_name;

  template <Named T>
    constexpr auto is_unnamed_v = is_unnamed<T>::value;
  template <Named T>
    constexpr auto get_name_v = get_name<T>::value;
  template <Named T>
    constexpr auto get_display_name_v = get_display_name<T>::value;

  // 21.11.4.5 Alias operations
  template <Alias T> struct get_aliased;

  template <Alias T>
    using get_aliased_t = typename get_aliased<T>::type;

  // 21.11.4.6 Type operations
  template <Typed T> struct get_type;
  template <Type T> struct get_reflected_type;
  template <Type T> struct is_enum;
  template <Type T> struct is_class;
  template <Type T> struct is_struct;
  template <Type T> struct is_union;

  template <Typed T>
    using get_type_t = typename get_type<T>::type;
  template <Type T>
    using get_reflected_type_t = typename get_reflected_type<T>::type;
  template <Type T>
    constexpr auto is_enum_v = is_enum<T>::value;
  template <Type T>
    constexpr auto is_class_v = is_class<T>::value;
  template <Type T>
    constexpr auto is_struct_v = is_struct<T>::value;
  template <Type T>
```

```
  constexpr auto is_union_v = is_union<T>::value;

// 21.11.4.7 Member operations
template <ScopeMember T> struct get_scope;
template <RecordMember T> struct is_public<T>;
template <RecordMember T> struct is_protected<T>;
template <RecordMember T> struct is_private<T>;

template <ScopeMember T>
  using get_scope_t = typename get_scope<T>::type;

// 21.11.4.8 Record operations
template <Record T> struct get_public_data_members;
template <Record T> struct get_accessible_data_members;
template <Record T> struct get_data_members;
template <Record T> struct get_public_member_types;
template <Record T> struct get_accessible_member_types;
template <Record T> struct get_member_types;
template <Class T> struct get_public_base_classes;
template <Class T> struct get_accessible_base_classes;
template <Class T> struct get_base_classes;
template <Class T> struct is_final;

template <Record T>
  using get_public_data_members_t = typename get_public_data_members<T>::type;
template <Record T>
  using get_accessible_data_members_t = typename get_accessible_data_members<T>::type;
template <Record T>
  using get_data_members_t = typename get_data_members<T>::type;
template <Record T>
  using get_public_member_types_t = typename get_public_member_types<T>::type;
template <Record T>
  using get_accessible_member_types_t = typename get_accessible_member_types<T>::type;
template <Record T>
  using get_member_types_t = typename get_member_types<T>::type;
template <Class T>
  using get_public_base_classes_t = typename get_public_base_classes<T>::type;
template <Class T>
  using get_accessible_base_classes_t = typename get_accessible_base_classes<T>::type;
template <Class T>
  using get_base_classes_t = typename get_base_classes<T>::type;
template <Class T>
  constexpr auto is_final_v = is_final<T>::value;

// 21.11.4.9 Enum operations
template <Enum T> struct is_scoped_enum;
template <Enum T> struct get_enumerators;
template <Enum T> struct get_underlying_type;

template <Enum T>
  constexpr auto is_scoped_enum_v = is_scoped_enum<T>::value;
template <Enum T>
  using get_enumerators_t = typename get_enumerators<T>::type;
template <Enum T>
  using get_underlying_type_t = typename get_underlying_type<T>::type;

// 21.11.4.10 Value operations
template <Constant T> struct get_constant;
template <Variable T> struct is_constexpr;
template <Variable T> struct is_static;
template <Variable T> struct get_pointer;

template <Constant T>
  constexpr auto get_constant_v = get_constant<T>::value;
template <Variable T>
  constexpr auto is_constexpr_v = is_constexpr<T>::value;
template <Variable T>
  constexpr auto is_static_v = is_static<T>::value;
template <Variable T>
  const auto get_pointer_v = get_pointer<T>::value;

// 21.11.4.11 Base operations
template <Base T> struct get_class;
template <Base T> struct is_virtual;
template <Base T> struct is_public<T>;
template <Base T> struct is_protected<T>;
template <Base T> struct is_private<T>;

template <Base T>
  using get_class_t = typename get_class<T>::type;
template <Base T>
  constexpr auto is_virtual_v = is_virtual<T>::value;

// 21.11.4.12 Namespace operations
template <Namespace T> struct is_inline;

template <Namespace T>
  constexpr auto is_inline_v = is_inline<T>::value;

} // inline namespace v1
} // namespace std::experimental::reflect
```

21.11.3 Concepts for meta-object types [reflect.concepts]

The operations on meta-object types defined here require meta-object types to satisfy certain concepts ([dcl.spec.concept]). These concepts are also used to specify the result type for *TransformationTrait* type transformations that yield meta-object types.

21.11.3.1 Concept `Object` [reflect.concepts.object]

```
template <class T> concept Object = see below;
```

`Object<T>` is satisfied if and only if `T` is a meta-object type, as generated by the `reflexpr` operator or any of the meta-object operations that in turn generate meta-object types.

21.11.3.2 Concept `ObjectSequence` [reflect.concepts.objseq]

```
template <class T> concept ObjectSequence = see below;
```

`ObjectSequence<T>` is satisfied if and only if `T` is a sequence of `Object`s, generated by a meta-object operation.

21.11.3.3 Concept `Named` [reflect.concepts.named]

```
template <class T> concept Named = see below;
```

`Named<T>` is satisfied if and only if `T` is an `Object` with an associated (possibly empty) name.

21.11.3.4 Concept `Alias` [reflect.concepts.alias]

```
template <class T> concept Alias = Named<T> && see below;
```

`Alias<T>` is satisfied if and only if `T` is a `Named` that reflects a `typedef` declaration, an *alias-declaration*, a *namespace-alias*, a template *type-parameter*, a *decltype-specifier*, or a declaration introduced by a *using-declaration*. Any such `T` also satisfies `ScopeMember`; its scope is the scope that the alias was injected into. [*Example:*

```
namespace N {
   struct A;
}
namespace M {
   using X = N::A;
}
using M_X_t = reflexpr(M::X);
using M_X_scope_t = get_scope_t<M_X_t>;
```

The scope reflected by `M_X_scope_t` is `M`, not `N`. — *end example*] Except for the type represented by the `reflexpr` operator, `Alias` properties resulting from type transformations (21.11.4) are not retained.

21.11.3.5 Concept `RecordMember` [reflect.concepts.recordmember]

```
template <class T> concept RecordMember = see below;
```

`RecordMember<T>` is satisfied if and only if `T` reflects a *member-declaration*. Any such `T` also satisfies `ScopeMember`.

21.11.3.6 Concept `Enumerator` [reflect.concepts.enumerator]

```
template <class T> concept Enumerator = see below;
```

`Enumerator<T>` is satisfied if and only if `T` reflects an enumerator. Any such `T` also satisfies `Typed` and `ScopeMember`; the `Scope` of an `Enumerator` is its type also for enumerations that are unscoped enumeration types.

21.11.3.7 Concept `Variable` [reflect.concepts.variable]

```
template <class T> concept Variable = see below;
```

`Variable<T>` is satisfied if and only if `T` reflects a variable or non-static data member. Any such `T` also satisfies `Typed`.

21.11.3.8 Concept `ScopeMember` [reflect.concepts.scopemember]

```
template <class T> concept ScopeMember = see below;
```

`ScopeMember<T>` is satisfied if and only if `T` satisfies `RecordMember`, `Enumerator`, or `Variable`, or if `T` reflects a namespace that is not the global namespace. Any such `T` also satisfies `Named`. The scope of members of an unnamed union is the unnamed union; the scope of enumerators is their type.

21.11.3.9 Concept `Typed` [reflect.concepts.typed]

```
template <class T> concept Typed = Variable<T> || Constant<T>;
```

`Typed<T>` is satisfied if and only if `T` reflects a variable or enumerator. Any such `T` also satisfies `Named`.

21.11.3.10 Concept `Namespace` [reflect.concepts.namespace]

```
template <class T> concept Namespace = see below;
```

`Namespace<T>` is satisfied if and only if `T` reflects a namespace (including the global namespace). Any such `T` also satisfies `Scope`. Any such `T` that does not reflect the global namespace also satisfies `ScopeMember`.

21.11.3.11 Concept `GlobalScope` [reflect.concepts.globalscope]

```
template <class T> concept GlobalScope = see below;
```

`GlobalScope<T>` is satisfied if and only if `T` reflects the global namespace. Any such `T` also satisfies `Namespace`; it does not satisfy `ScopeMember`.

21.11.3.12 Concept `Class` [reflect.concepts.class]

```
template <class T> concept Class = see below;
```

`Class<T>` is satisfied if and only if `T` reflects a non-union class type. Any such `T` also satisfies `Record`.

21.11.3.13 Concept `Enum` [reflect.concepts.enum]

```
template <class T> concept Enum = see below;
```

`Enum<T>` is satisfied if and only if `T` reflects an enumeration type. Any such `T` also satisfies `Type` and `Scope`.

21.11.3.14 Concept `Record` [reflect.concepts.record]

```
template <class T> concept Record = see below;
```

`Record<T>` is satisfied if and only if `T` reflects a class type. Any such `T` also satisfies `Type` and `Scope`.

21.11.3.15 Concept `Scope` [reflect.concepts.scope]

```
template <class T> concept Scope = Namespace<T> || Record<T> || Enum<T>;
```

`Scope<T>` is satisfied if and only if `T` reflects a namespace (including the global namespace), class, or enumeration. Any such `T` that does not reflect the global namespace also satisfies `ScopeMember`.

21.11.3.16 Concept `Type` [reflect.concepts.type]

```
template <class T> concept Type = see below;
```

`Type<T>` is satisfied if and only if `T` reflects a type. Any such `T` also satisfies `Named` and `ScopeMember`.

21.11.3.17 Concept `Constant` [reflect.concepts.const]

```
template <class T> concept Constant = see below;
```

`Constant<T>` is satisfied if and only if `T` reflects a constant expression ([expr.const]). Any such `T` also satisfies `ScopeMember` and `Typed`.

21.11.3.18 Concept `Base` [reflect.concepts.base]

```
template <class T> concept Base = see below;
```

`Base<T>` is satisfied if and only if `T` reflects a direct base class, as returned by the template `get_base_classes`.

21.11.4 Meta-object Operations [reflect.ops]

A meta-object operation extracts information from meta-object types. It is a class template taking one or more

arguments, at least one of which models the `Object` concept. The result of a meta-object operation can be either a constant expression ([expr.const]) or a type.

21.11.4.1 Multi-concept operations [reflect.ops.over]

```
template <class T> struct is_public;
template <class T> struct is_protected;
template <class T> struct is_private;
```

These meta-object operations are applicable to both `RecordMember` and `Base`. The generic templates do not have a definition. When multiple concepts implement the same meta-object operation, its template will be partially specialized for the concepts implementing the operation. [*Note:* For these overloaded operations, any meta-object type will always satisfy at most one of the concepts that the operation is applicable to. — *end note*]

[*Example:* An operation `OP` applicable to concepts `A` and `B` can be defined as follows:

```
template <class T> concept A = is_signed_v<T>;
template <class T> concept B = is_class_v<T>;
template <class T> struct OP; // undefined
template <A T> struct OP<T> {...};
template <B T> struct OP<T> {...};
```

— *end example*]

21.11.4.2 Object operations [reflect.ops.object]

```
template <Object T1, Object T2> struct reflects_same;
```

All specializations of `reflects_same<T1, T2>` shall meet the `BinaryTypeTrait` requirements ([meta.rqmts]), with a base characteristic of `true_type` if

- `T1` and `T2` reflect the same alias, or
- neither `T1` nor `T2` reflect an alias and `T1` and `T2` reflect the same entity;

otherwise, with a base characteristic of `false_type`.

[*Example:* With

```
class A;
using a0 = reflexpr(A);
using a1 = reflexpr(A);
class A {};
using a2 = reflexpr(A);
constexpr bool b1 = is_same_v<a0, a1>; // unspecified value
constexpr bool b2 = reflects_same_v<a0, a1>; // true
constexpr bool b3 = reflects_same_v<a0, a2>; // true

struct C { };
using C1 = C;
using C2 = C;
constexpr bool b4 = reflects_same_v<reflexpr(C1), reflexpr(C2)>; // false
```

— *end example*]

```
template <class T> struct get_source_line;
template <class T> struct get_source_column;
```

All specializations of above templates shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]) with a base characteristic of `integral_constant<uint_least32_t>` and a value of the presumed line number ([cpp.predefined]) (for `get_source_line<T>`) and an implementation-defined value representing some offset from the start of the line (for `get_source_column<T>`) of the most recent declaration of the entity or typedef described by `T`.

```
template <class T> struct get_source_file_name;
```

All specializations of `get_source_file_name<T>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]) with a static data member named `value` of type `const char (&)[N]`, referencing a static, constant expression character array (NTBS) of length `N`, as if declared as `static constexpr char STR[N] = ...;`. The value of the NTBS is the presumed name of the source file ([cpp.predefined]) of the most recent declaration of the entity or typedef described by `T`.

21.11.4.3 ObjectSequence operations [reflect.ops.objseq]

```
template <ObjectSequence S> struct get_size;
```

All specializations of `get_size<S>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]) with a base characteristic of `integral_constant<size_t, N>`, where `N` is the number of elements in the object sequence.

```
template <size_t I, ObjectSequence S> struct get_element;
```

Remarks: All specializations of `get_element<I, S>` shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` corresponds to the I^{th} element `Object` in `S`, where the indexing is zero-based.

```
template <template <class...> class Tpl, ObjectSequence S>
  struct unpack_sequence;
```

Remarks: All specializations of `unpack_sequence<Tpl, S>` shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is an alias to the template `Tpl` specialized with the types in `S`.

21.11.4.4 Named operations [reflect.ops.named]

```
template <Named T> struct is_unnamed;
template <Named T> struct get_name;
template <Named T> struct get_display_name;
```

All specializations of `is_unnamed<T>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]) with a base characteristic as specified below.

All specializations of `get_name` and `get_display_name` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]) with a static data member named `value` of type `const char (&)[N]`, referencing a static, constant expression character array (NTBS) of length `N`, as if declared as `static constexpr char STR[N] = ...;`.

- For `T` reflecting an unnamed entity, the string's value is the empty string.
- For `T` reflecting a *decltype-specifier*, the string's value is the empty string for `get_name<T>` and implementation-defined for `get_display_name<T>`.
- For `T` reflecting an array, pointer, reference of function type, or a *cv*-qualified type, the string's value is the empty string for `get_name<T>` and implementation-defined for `get_display_name<T>`.
- In the following cases, the string's value is implementation-defined for `get_display_name<T>` and has the following value for `get_name<T>`:
  - for `T` reflecting an `Alias`, the unqualified name of the aliasing declaration: the identifier introduced by a *type-parameter* or a type name introduced by a *using-declaration*, alias;
  - for `T` reflecting a specialization of a class template, its *template-name*;
  - for `T` reflecting a class type, its *class-name*;
  - for `T` reflecting a namespace, its *namespace-name*;
  - for `T` reflecting an enumeration type, its *enum-name*;
  - for `T` reflecting all other *simple-type-specifiers*, the name stated in the "Type" column of Table 9 in ([dcl.type.simple]);
  - for `T` reflecting a variable, its unqualified name;
  - for `T` reflecting an enumerator, its unqualified name;
  - for `T` reflecting a class data member, its unqualified name.
- In all other cases, the string's value is the empty string for `get_name<T>` and implementation-defined for `get_display_name<T>`.

[*Note:* With

```
namespace n { template <class T> class A; }
using a_m = reflexpr(n::A<int>);
```

the value of `get_name_v<a_m>` is `"A"` while the value of `get_display_name_v<a_m>` might be `"n::A<int>"`. — *end note*]

The base characteristic of `is_unnamed<T>` is `true_type` if the value of `get_name_v<T>` is the empty string, otherwise it is `false_type`.

21.11.4.5 Alias operations [reflect.ops.alias]

```
template <Alias T> struct get_aliased;
```

All specializations of `get_aliased<T>` shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is the `Named` meta-object type reflecting

- the redefined name, if `T` reflects an alias;
- the template specialization's template argument type, if `T` reflects a template *type-parameter*;

- the original declaration introduced by a *using-declaration*;
- the aliased namespace of a *namespace-alias*;
- the type denoted by the *decltype-specifier*.

The nested type named `type` is not an `Alias`; instead, it is reflecting the underlying non-`Alias` entity.

[*Example:* For

```
using i0 = int; using i1 = i0;
```

`get_aliased_t<reflexpr(i1)>` reflects `int`. — *end example*]

21.11.4.6 Type operations [reflect.ops.type]

```
template <Typed T> struct get_type;
```

All specializations of `get_type<T>` shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is the `Type` reflecting the type of the entity reflected by `T`.

[*Example:* For

```
int v; using v_m = reflexpr(v);
```

`get_type_t<v_m>` reflects `int`. — *end example*]

If the entity reflected by `T` is a static data member that is declared to have a type array of unknown bound in the class definition, possible specifications of the array bound will only be accessible when the *reflexpr-operand* is the data member.

[*Note:* For

```
struct C {
   static int arr[17][];
};
int C::arr[17][42];
using C1 = get_type_t<get_element_t<0, get_data_members_t<reflexpr(C)>>>;
using C2 = get_type_t<reflexpr(C::arr)>;
```

`C1` will reflect `int[17][]` while `C2` will reflect `int[17][42]`. — *end note*]

```
template <Type T> struct get_reflected_type;
```

All specializations of `get_reflected_type<T>` shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is the type reflected by `T`.

[*Example:* For

```
using int_m = reflexpr(int);
get_reflected_type_t<int_m> x; // x is of type int
```

— *end example*]

```
template <Type T> struct is_enum;
template <Type T> struct is_union;
```

All specializations of `is_enum<T>` and `is_union<T>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]). If `T` reflects an enumeration type (a union), the base characteristic of `is_enum<T>` (`is_union<T>`) is `true_type`, otherwise it is `false_type`.

```
template <Type T> struct is_class;
template <Type T> struct is_struct;
```

All specializations of these templates shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]). If `T` reflects a class with *class-key* `class` (for `is_class<T>`) or `struct` (for `is_struct<T>`), the base characteristic of the respective template specialization is `true_type`, otherwise it is `false_type`. If the same class has redeclarations with both *class-key* `class` and *class-key* `struct`, the base characteristic of the template specialization of exactly one of `is_class<T>` and `is_struct<T>` can be `true_type`, the other template specialization is `false_type`; the actual choice of value is unspecified.

21.11.4.7 Member operations [reflect.ops.member]

A specialization of any of these templates with a meta-object type that is reflecting an incomplete type renders the program ill-formed. Such errors are not in the immediate context ([temp.deduct]).

```
template <ScopeMember T> struct get_scope;
```

All specializations of `get_scope<T>` shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is the `Scope` reflecting a scope *S*. With *ST* being the scope of the declaration of the entity or typedef reflected by `T`, *S* is found as the innermost scope enclosing *ST* that is either a namespace scope (including global scope), class scope, or enumeration scope.

```
template <RecordMember T> struct is_public<T>;
template <RecordMember T> struct is_protected<T>;
template <RecordMember T> struct is_private<T>;
```

All specializations of these partial template specializations shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]). If `T` reflects a public member (for `is_public`), protected member (for `is_protected`), or private member (for `is_private`), the base characteristic of the respective template specialization is `true_type`, otherwise it is `false_type`.

21.11.4.8 Record operations [reflect.ops.record]

A specialization of any of these templates with a meta-object type that is reflecting an incomplete type renders the program ill-formed. Such errors are not in the immediate context ([temp.deduct]).

```
template <Record T> struct get_public_data_members;
template <Record T> struct get_accessible_data_members;
template <Record T> struct get_data_members;
```

All specializations of these templates shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is an alias to an `ObjectSequence` specialized with `RecordMember` types that reflect the following subset of data members of the class reflected by `T`:

- for `get_data_members`, all data members.
- for `get_public_data_members`, all public data members;
- for `get_accessible_data_members`, all data members that are accessible from the scope of the invocation of `reflexpr` which (directly or indirectly) generated `T`.

The order of the elements in the `ObjectSequence` is the order of the declaration of the data members in the class reflected by `T`.

Remarks: The program is ill-formed if `T` reflects a closure type.

```
template <Record T> struct get_public_member_types;
template <Record T> struct get_accessible_member_types;
template <Record T> struct get_member_types;
```

All specializations of these templates shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is an alias to an `ObjectSequence` specialized with `Type` types that reflect the following subset of types declared in the class reflected by `T`:

- for `get_member_types`, all nested class types, enum types, or member typedefs.
- for `get_public_member_types`, all public nested class types, enum types, or member typedefs;
- for `get_accessible_member_types`, all nested class types, enum types, or member typedefs that are accessible from the scope of the invocation of `reflexpr` which (directly or indirectly) generated `T`.

The order of the elements in the `ObjectSequence` is the order of the first declaration of the types in the class reflected by `T`.

Remarks: The program is ill-formed if `T` reflects a closure type.

```
template <Class T> struct get_public_base_classes;
template <Class T> struct get_accessible_base_classes;
template <Class T> struct get_base_classes;
```

All specializations of these templates shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is an alias to an `ObjectSequence` specialized with `Base` types that reflect the following subset of base classes of the class reflected by `T`:

- for `get_base_classes`, all direct base classes;
- for `get_public_base_classes`, all public direct base classes;

- for `get_accessible_base_classes`, all direct base classes whose public members are accessible from the scope of the invocation of `reflexpr` which (directly or indirectly) generated `T`.

The order of the elements in the `ObjectSequence` is the order of the declaration of the base classes in the class reflected by `T`.

Remarks: The program is ill-formed if `T` reflects a closure type.

```
template <Class T> struct is_final;
```

All specializations of `is_final<T>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]). If `T` reflects a class that is marked with the *class-virt-specifier* `final`, the base characteristic of the respective template specialization is `true_type`, otherwise it is `false_type`.

21.11.4.9 Enum operations [reflect.ops.enum]

```
template <Enum T> struct is_scoped_enum;
```

All specializations of `is_scoped_enum<T>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]). If `T` reflects a *scoped enumeration*, the base characteristic of the respective template specialization is `true_type`, otherwise it is `false_type`.

```
template <Enum T> struct get_enumerators;
```

All specializations of `get_enumerators<T>` shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is an alias to an `ObjectSequence` specialized with `Enumerator` types that reflect the enumerators of the enumeration type reflected by `T`.

Remarks: A specialization of this template with a meta-object type that is reflecting an incomplete type renders the program ill-formed. Such errors are not in the immediate context ([temp.deduct]).

```
template <Enum T> struct get_underlying_type;
```

All specializations of `get_underlying_type<T>` shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is an alias to a meta-object type that reflects the underlying type (10.2) of the enumeration reflected by `T`.

Remarks: A specialization of this template with a meta-object type that is reflecting an incomplete type renders the program ill-formed. Such errors are not in the immediate context ([temp.deduct]).

21.11.4.10 Value operations [reflect.ops.value]

```
template <Constant T> struct get_constant;
```

All specializations of `get_constant<T>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]). It has a static data member named `value` whose type and value are those of the constant expression of the constant reflected by `T`.

```
template <Variable T> struct is_constexpr;
```

All specializations of `is_constexpr<T>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]). If `T` reflects a variable declared with the *decl-specifier* `constexpr`, the base characteristic of the respective template specialization is `true_type`, otherwise it is `false_type`.

```
template <Variable T> struct is_static;
```

All specializations of `is_static<T>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]). If `T` reflects a variable with static storage duration, the base characteristic of the respective template specialization is `true_type`, otherwise it is `false_type`.

```
template <Variable T> struct get_pointer;
```

All specializations of `get_pointer<T>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]), with a static data member named `value` of type `X` and value `x`, where

- for variables with static storage duration: `X` is `add_pointer<Y>`, where `Y` is the type of the variable reflected by `T` and `x` is the address of that variable; otherwise,
- `X` is the pointer-to-member type of the member variable reflected by `T` and `x` a pointer to the member.

21.11.4.11 Base operations **[reflect.ops.derived]**

A specialization of any of these templates with a meta-object type that is reflecting an incomplete type renders the program ill-formed. Such errors are not in the immediate context ([temp.deduct]).

```
template <Base T> struct get_class;
```

All specializations of `get_class<T>` shall meet the `TransformationTrait` requirements ([meta.rqmts]). The nested type named `type` is an alias to `reflexpr(`*X*`)`, where *X* is the base class reflected by `T`.

```
template <Base T> struct is_virtual;
template <Base T> struct is_public<T>;
template <Base T> struct is_protected<T>;
template <Base T> struct is_private<T>;
```

All specializations of the template and of these partial template specializations shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]). If `T` reflects a direct base class with the `virtual` specifier (for `is_virtual`), with the `public` specifier or with an assumed (see C++ [class.access.base]) `public` specifier (for `is_public`), with the `protected` specifier (for `is_protected`), or with the `private` specifier or with an assumed `private` specifier (for `is_private`), then the base characteristic of the respective template specialization is `true_type`, otherwise it is `false_type`.

21.11.4.12 Namespace operations **[reflect.ops.namespace]**

```
template <Namespace T> struct is_inline;
```

All specializations of `is_inline<T>` shall meet the `UnaryTypeTrait` requirements ([meta.rqmts]). If `T` reflects an inline namespace, the base characteristic of the template specialization is `true_type`, otherwise it is `false_type`.

Revision history

Revision 1 (N3996)

Describes the method of static reflection by means of compiler-generated unnamed types. Introduces the first version of the metaobject concepts and some possibilities of their implementation. Also includes discussion about the motivation and the design rationale for the proposal.

Revision 2 (N4111)

Refines the metaobject concepts and introduces a concrete implementation of their interface by the means of templates similar to the standard type traits. Describes some additions to the standard library (mostly meta-programming utilities), which simplify the use of the metaobjects. Answers some questions from the discussion about N3996 and expands the design rationale.

Revision 3 (N4451)

Incorporates the feedback from the discussion about N4111 at the Urbana meeting, most notably reduces the set of metaobject concepts and refines their definitions, removes some of the additions to the standard library added in the previous revisions. Adds context-dependent reflection.

Revision 4 (P0194R0)

Further refines the concepts from N4111; prefixes the names of the metaobject operations with `get_`, adds new operations, replaces the metaobject category tags with new metaobject traits. Introduces a nested namespace `std::reflect` which contains most of the reflection-related additions to the standard library. Rephrases definition of meta objects using Concepts Lite. Specifies the reflection operator name — `reflexpr`. Introduces an experimental implementation of the reflection operator in clang. Drops the context-dependent reflection from N4111 (will be re-introduced later).

Revision 5 (P0194R1)

Dropped all metaobject traits except `is_metaobject`. All metaobject classification is now done by using the concepts.

The `meta::Scoped` concept has been renamed to `meta::ScopeMember`. The `meta::Constant` and `meta::Specifier` concepts, and

several new operations have been added.

The aliases for the operation templates returning metaobjects had previously the `_t` suffix; this has been changed to the `_m` suffix.

Revision 6 (P0194R2)

The following concepts from P0194R1 were dropped in order to simplify the proposal: `meta::Linkable`, `meta::Enum`, `meta::DataMember`, `meta::MemberType`, `meta::EnumClass`, `meta::TypeAlias` and `meta::NamespaceAlias`.

The following concepts were added to the proposal: `meta::TagType`, `meta::Record`, `meta::Enumerator`.

Unlike in the previous proposal, metaobjects reflecting unnamed entities - the global scope, unnamed namespaces and classes, etc. *do* conform to the `meta::Named` concept and implement the name-returning operations.

Unlike in the previous proposal, metaobjects reflecting the global scope *do* conform to the `meta::ScopeMember` concept and the `meta::get_scope` operation. For arguments reflecting the global scope returns a metaobject reflecting the global scope (i.e. the global scope is its own scope).

Metaobjects reflecting built-in types and types like pointers, references, arrays, etc. now don't have a scope (i.e. they do not conform to the `meta::ScopeMember` concept).

We have added a different mechanism for distinguishing between unscoped and scoped `enum`s - the `meta::is_scoped_enum` operation. Unlike in the previous proposal, `meta::Enum` reflecting a unscoped enum *is* a `meta::Scope`.

We now allow the default construction and copy construction of values of metaobject types.

Direct reflection of class data members, member types and type aliases, enumerators and global scope/namespace-level variables has been added.

The typedef (type-alias) reflection has been simplified based on the feedback from Oulu. Previously we required reflection to be aware about all aliases in a *"chain"* even in the context of templates.

The mechanism for enumerating public-only vs. all (including non-public ones) class members has been changed. Now the *"basic"* operations like `meta::get_data_members`, `meta::get_member_types`, etc. return all members, and the `meta::get_public_data_members`, `meta::get_public_member_types`, return only the public class members.

Revision 7 (P0194R3)

Major wording update, clarifying the behavior for instance on dependent names and function-local declarations. All examples but those required for the wording are now found in the design discussion paper, P0385.

Rename header, namespace from `meta` to `reflect`. Target a TS.

Use of function-style concepts, following the example set by the Ranges TS, for consistency reasons. The name of the reflection operator was changed to `$reflect`; the *basic source character set* was adjusted accordingly.

In addition to `get_public_base_classes`, `get_public_data_members` and `get_public_member_types` and their sibling without `public`, a new sibling `get_accessible_...` was added. The `Specifier`, `TagType` and `Reversible` concepts were removed. So was the `is_metaobject` trait. The type of meta-object operations "returning" strings have been updated. The order of template parameters for `MetaSequence` operations has been matched to similar operations of `tuple`. The suffix `_m` was reverted to `_t`. `$reflect()` with an empty operand is disallowed.

Revision 8 (P0194R4)

As requested by EWG in Kona, the operator was renamed back to `reflexpr`; no need to change the source character set anymore. `get_base_name` is now called `get_name` and returns an empty string on pointer, references, arrays and *cv*-qualified types.

Revision 9 (P0194R5)

Switched concept definitions to use variable concepts, unconstrained them (constrained concepts don't exist). Proper wording for the language part, based on a mixture of C++17 and the Concepts TS). Moved [meta] to section 20.15.

Revision 10 (P0194R6)

A meta-object type is now incomplete (instead of trivial). Names of function types are now specified to be empty. A name found through a using directives does not create an `Alias` anymore. `GlobalScope` is not a `ScopeMember` anymore. Using declarations of members are now supported. The behavior of `is_class` versus `is_struct` is now clarified, especially in the case of conflicting redeclarations. Added `get_underlying_type<Enum>`. A structured binding now behaves as a `Variable`. Variables wil an initially declared type of array of unknown bound will only see a complete bound if reflected through the member (in contrast to a reflection through the class). Do not reference `source_location` to not create a dependency on LibFunv2.

References

1. *Static reflection. Rationale, design and evolution.* P0385

2. *Static reflection in a nutshell.* P0578